

Honey Chain: commands for the jury

How to show the real database, each company's private database and the blockchain. Everything runs on this computer: a PostgreSQL server and a local blockchain, both started by one command.

1. Start and connect

What	Command / value
Start everything (from E:\vs\SIH)	npm start (wait for "Local Postgres ready" and "[chain] connected")
Website (verify a jar)	http://localhost:5175
Keeper app / Admin app	http://localhost:5173 / http://localhost:5174
DBeaver connection	Host 127.0.0.1 Port 54329 Database honeychain User postgres Password postgres
One-page inspector (2nd terminal)	npm run inspect then open http://localhost:5177
Stop everything	Ctrl+C in the npm start window (the database shuts down cleanly)

In DBeaver open a SQL editor with Ctrl+J and run one query at a time with Ctrl+Enter. Left tree: honeychain > Databases > honeychain > Schemas > public > Tables.

Officer logins (also shown on the login pages): kvic.head / kvic@12345, state.tn / state@12345, region.madurai / region@12345.

2. Registered companies and their own databases

The schema "public" is the common database. Every company also gets its own private schema (co_org_...), a separate set of tables that the admin app never reads.

How many companies are registered

```
SELECT status, COUNT(*) AS companies FROM organizations GROUP BY status;
```

The companies with their details and login username

```
SELECT o.organization_code, o.legal_name, o.status, o.state, o.region, o.fssai_license,
       u.username, s.email, s.phone, o.created_at
FROM organizations o
JOIN users u ON u.id = o.owner_user_id
LEFT JOIN user_settings s ON s.user_id = u.id
ORDER BY o.id;
```

Which private database belongs to which company

```
SELECT legal_name, organization_code,
       'co_' || lower(replace(organization_code, '-', '_')) AS private_schema
FROM organizations ORDER BY id;
```

The private databases that exist (their count = number of companies)

```
SELECT schema_name FROM information_schema.schemata
WHERE schema_name LIKE 'co\_%' ORDER BY 1;
```

Every table of every company, with its number of rows

```
SELECT table_schema, table_name,
       (xpath('/row/c/text()', query_to_xml(format('select count(*) as c from %I.%I',
       table_schema, table_name), false, true, ''))[1]::text::int AS rows
FROM information_schema.tables
WHERE table_schema LIKE 'co\_%' AND table_type = 'BASE TABLE'
ORDER BY 1, 2;
```

Open one company's private data (replace the schema name with one from above)

```
SELECT * FROM co_org_2026_4bcc391a.harvests;
SELECT * FROM co_org_2026_4bcc391a.finance_entries;
SELECT * FROM co_org_2026_4bcc391a.buyers;
```

Say: Prices, buyers and finance live only in the company's own schema.

What the government side sees of a company's money: one total per month

```
SELECT o.legal_name, m.month, m.income_inr
FROM monthly_reports m JOIN organizations o ON o.id = m.org_id ORDER BY 1, 2;
```

Say: Compare with the finance_entries of the private schema: the admin only ever gets this monthly total.

Logins and passwords

Passwords are never stored in readable form, only as a bcrypt hash (starts with \$2b\$). This is on purpose: nobody with database access can read a password.

```
SELECT username, role, left(password, 7) || '...' AS password_hash
FROM users WHERE role = 'BEEKEEPER';
```

3. The blockchain as SQL tables (audit ledger)

Every action (registration, harvest, lab result, jar QR codes, hand-over) becomes a row in traceability_events. Events are sealed into signed blocks in chain_blocks. Each row stores the hash of the row before it, so changing any old row breaks every later link.

The latest blocks

```
SELECT height, tx_count, substr(block_hash,1,16) AS hash,
       substr(prev_hash,1,16) AS prev, validator_id
FROM chain_blocks ORDER BY height DESC LIMIT 10;
```

Say: Each block's prev equals the previous block's hash. Block 0 is the genesis block.

The latest events (the transactions)

```
SELECT id, event_type, entity_id, substr(prev_hash,1,12) AS prev,
       substr(current_hash,1,12) AS hash
FROM traceability_events ORDER BY id DESC LIMIT 10;
```

The full history of one batch (replace the batch code)

```
SELECT id, event_type, created_at, substr(current_hash,1,12) AS hash
FROM traceability_events WHERE entity_id = 'HC-2026-0001' ORDER BY id;
```

Check the whole chain in SQL: both answers must be 0

```
SELECT COUNT(*) AS broken_event_links FROM (
  SELECT prev_hash, LAG(current_hash) OVER (ORDER BY id) AS expected
  FROM traceability_events) t WHERE expected IS NOT NULL AND prev_hash <> expected;

SELECT COUNT(*) AS broken_block_links FROM (
  SELECT prev_hash, LAG(block_hash) OVER (ORDER BY height) AS expected
  FROM chain_blocks) t WHERE expected IS NOT NULL AND prev_hash <> expected;
```

Try to cheat: both are refused by the database

```
UPDATE traceability_events SET payload_json = '{}' WHERE id = 1;
DELETE FROM chain_blocks WHERE height = 0;
```

Say: "traceability ledger is immutable" and "blockchain is immutable". Database locks reject any update, delete or truncate.

The locks themselves

```
SELECT tgrelid::regclass AS table_name, tgname AS lock
FROM pg_trigger WHERE NOT tgisinternal ORDER BY 1, 2;
```

Honest note for questions: the database blocks edits, but whoever owns the database password could remove a lock. They cannot hide it: the hashes and the validator's signatures would no longer match, and Admin > Blockchain > "Verify every block now" names the exact block. That is tamper-evident, single-validator (proof of authority), not a decentralised network.

4. The smart contract (second blockchain layer)

A Solidity contract on the local chain records who registered each batch (the harvester's signature) and who certified it (a second, officer signature), the jar Merkle roots, the loose-sale limit and the head of the ledger above. The API sends these as transactions; the table below is the queue with each transaction hash.

Transactions sent to the contract

```
SELECT id, kind, ref, status, block_number, tx_hash
FROM chain_outbox ORDER BY id DESC LIMIT 20;
```

Are they all confirmed?

```
SELECT status, COUNT(*) AS n FROM chain_outbox GROUP BY status;
```

Which blockchain account belongs to which user (addresses only, keys are encrypted)

```
SELECT u.username, w.address
FROM chain_wallets w JOIN users u ON u.id = w.user_id;
```

Show the contract itself: Admin app > Blockchain > "Smart contract" tab, or the inspector page (npm run inspect) > "Smart contract" tab: contract address, block number, decoded events, transactions. On the public website, scanning a jar shows an on-chain proof card.

5. Other commands

Command (folder)	What it does
npm run db:verify (APICtech/backend)	Reads the common schema and every company schema and prints tables and rows
npm run db:backup (APICtech/backend)	Portable export, one file per table, with a SHA-256 manifest (also automatic daily)
npm run db:check -- <export-folder>	Re-checks an export against its manifest
npm run db:import-sqlite (APICtech/backend)	One-time: moves data from the earlier SQLite version into PostgreSQL (stop npm start first)
npm run jury (E:\vs\SIH)	Writes the inspector as one offline HTML file
npm test (blockchain)	Runs the 7 smart-contract tests

6. Ready-made answers

Where is the data stored?

In PostgreSQL 18 on this computer. The common tables are in the schema "public"; every company has its own private schema.

Can the government see a company's finances?

No. Only a monthly income total is shared. Everything else stays in the company's private schema.

Can the blockchain be changed?

Not by the application, and the database refuses edits. If someone bypasses that, the hash chain and signatures expose it and the Admin page names the block.

What is on the smart contract?

Batch registration (harvester signature), certification (second, officer signature), jar Merkle roots, sale limits and the ledger head. Private data is never put on it.

Is it a public blockchain?

The demo runs a local chain. It can be pointed at the Polygon Amoy test network by setting two settings; that has not been tested yet.